

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



SEARCHING

Oleh:
Aufa Thariq Akram **2510817310014**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Searching ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Source Code	3
B. Output Program	8
C. Pembahasan	10
LINK GITHUB	15

DAFTAR GAMBAR

Gambar 1 Tampilan Awal Program	8
Gambar 2 Sequential Searching dengan Input 7	8
Gambar 3 Binary Searching Dengan Ukuran Vektor 19 dan Mencari Angka 71	9
Gambar 4 Penjelasan Perbedaan Sequential Searching dan Binary Searching.....	9
Gambar 5 Exit	9

DAFTAR TABEL

Tabel 1 Source Code Soal 1	3
----------------------------------	---

SOAL 1

Ketikkan source code berikut pada program IDE bahasa pemrograman C++
(Gabungkan 2 code berikut menjadi 1 file (Menu):

- Sequential Searching

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <time.h>
4
5  using namespace std;
6
7  int random(int bil)
8  {
9      int jumlah = rand() % bil;
10     return jumlah;
11 }
12
13 void randomize()
14 {
15     srand(time(NULL));
16 }
17
18 void clrscr()
19 {
20     system("cls");
21 }
22
23 int main()
24 {
25     clrscr();
26     int data[100];
27     int cari = 20;
28     int counter = 0;
29     int flag = 0;
30     int save;
31     randomize();
32     printf("generating 100 number . . .\n");
33     for (int i = 0; i < 100; i++)
34     {
35         data[i] = random(100) + 1;
36         printf("%d ", data[i]);
37     }
38     printf("\ndone.\n");
39
40     for (int i = 0; i < 100; i++)
41     {
42         if (data[i] == cari)
43         {
44             counter++;
45             flag = 1;
46             save = i;
47         }
48     }
49
50     if (flag == 1)
51     {
52         printf("Data ada, sebanyak %d!\n", counter);
53         printf("pada indeks ke-%d", save);
54     }
55     else
56     {
57         printf("Data tidak ada!\n");
58     }
59 }
60
```

- Binary Searching

```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n, kiri, kanan, tengah, temp, key;
7      bool ketemu = false;
8
9      cout << "Masukan jumlah data? ";
10     cin >> n;
11     int angka[n];
12     for (int i = 0; i < n; i++)
13     {
14         cout << "Angka ke - [" << i << "] : ";
15         cin >> angka[i];
16     }
17
18     for (int i = 0; i < n; i++)
19     {
20         for (int j = 0; j < n - 1; j++)
21         {
22             if (angka[j] > angka[j + 1])
23             {
24                 temp = angka[j];
25                 angka[j] = angka[j + 1];
26                 angka[j + 1] = temp;
27             }
28         }
29     }
30     cout << "\n-----\n";
31     cout << "Data yang telah diurutkan adalah:\n";
32     for (int i = 0; i < n; i++)
33     {
34         cout << angka[i] << " ";
35     }
36     cout << "\n-----\n";
37     cout << "Masukan angka yang dicari: ";
38     cin >> key;
39
40     kiri = 0;
41     kanan = n - 1;
42     while (kiri <= kanan)
43     {
44         tengah = (kiri + kanan) / 2;
45         if (key == angka[tengah])
46         {
47             ketemu = true;
48             break;
49         }
50         else if (key < angka[tengah])
51         {
52             kanan = tengah - 1;
53         }
54         else
55         {
56             kiri = tengah + 1;
57         }
58     }
59     if (ketemu == true)
60     {
61         cout << "Angka ditemukan! ";
62     }
63     else
64     {
65         cout << "Angka tidak ditemukan!";
66     }
67     return 0;
68 }

```

- Tampilan Menu Program:

```

Pilih menu
1. Sequential Searching
2. Binary Searching
3. Jelaskan Perbedaan Sequential Searching dan Binary Searching!
4. Exit
Pilih :

```

Jelaskan perbedaan Sequential Searching dan Binary Searching beserta kelebihan dan kekurangan masing-masing?

A. Source Code

Tabel 1 Source Code Soal 1

```
1  #include <iostream>
2  #include <random>
3  #include <vector>
4  #include <algorithm>
5  #include <termios.h>
6  #include <unistd.h>
7
8  using namespace std;
9
10 int getch() {
11     struct termios oldt, newt;
12     int ch;
13     tcgetattr(STDIN_FILENO, &oldt);
14     newt = oldt;
15     newt.c_lflag &= ~(ICANON | ECHO);
16     tcsetattr(STDIN_FILENO, TCSANOW, &newt);
17     ch = getchar();
18     tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
19     return ch;
20 }
21
22 void sequentialSearch(vector<int> &nums, int target)
23 {
24     int counter = 0;
25     int lastIndex = -1;
26
27     for (int i = 0; i < nums.size(); i++) {
28         if (nums[i] == target) {
29             counter++;
30             lastIndex = i;
31         }
32     }
33
34     if (counter > 0) {
35         cout << "Data ditemukan sebanyak " <<
36         counter << " kali.\n";
37         cout << "Kemunculan terakhir pada indeks ke-
38         " << lastIndex << ".\n";
39     } else {
40         cout << "Data tidak ditemukan!\n";
41     }
42 }
```



```

40
41 void binarySearch(vector<int> &nums, int target) {
42     sort(nums.begin(), nums.end());
43
44     cout << "Data setelah diurutkan:\n";
45     cout << "Data: ";
46     for (int val : nums) {
47         cout << val << " ";
48     }
49     cout << endl;
50
51     int kiri = 0, kanan = nums.size() - 1;
52     bool ketemu = false;
53
54     while (kiri <= kanan) {
55         int tengah = (kiri + kanan) / 2;
56         if (nums[tengah] == target) {
57             ketemu = true;
58             break;
59         } else if (target < nums[tengah]) {
60             kanan = tengah - 1;
61         } else {
62             kiri = tengah + 1;
63         }
64     }
65
66     if (ketemu) {
67         cout << "Angka ditemukan!" << endl;
68     } else {
69         cout << "Angka tidak ditemukan!" << endl;
70     }
71 }
72
73 void clearScreen() {
74     system("clear");
75 }
76
77 // MODIFIKASI: Fungsi explain() dirombak total
menggunakan gaya penjelasan baru
78 void explain() {
79     cout <<
"\n=====
=====\\n";
80     cout << "          KONSEP DASAR SEQUENTIAL & BINARY
SEARCHING\\n";

```

```

81         cout <<
"=====
=====\n\n";
82
83         cout << "[A] SEQUENTIAL SEARCH (Pencarian
Linear)\n";
84         cout << "Analogi: Seperti mencari sebuah buku
spesifik di dalam tumpukan\n";
85         cout << "          kardus yang berantakan dengan
mengeceknnya satu per satu.\n";
86         cout << "Cara Kerja:\n";
87         cout << "- Algoritma menyisir data dari elemen
paling awal (indeks 0)\n";
88         cout << "  sampai elemen paling akhir secara
berurutan.\n";
89         cout << "Kelebihan:\n";
90         cout << "  * Sangat fleksibel! Data boleh dalam
kondisi acak atau tidak teratur.\n";
91         cout << "Kekurangan:\n";
92         cout << "  * Sangat lambat untuk data berskala
besar (kompleksitas  $O(n)$ ).\n";
93         cout << "    Bayangkan mengecek 1 juta data satu
per satu dari awal.\n\n";
94
95         cout << "[B] BINARY SEARCH (Pencarian Bagi
Dua)\n";
96         cout << "Analogi: Seperti mencari nama orang di
dalam buku telepon.\n";
97         cout << "          Kita langsung buka bagian
tengah, lalu menebak apakah\n";
98         cout << "          namanya ada di paruh kiri atau
kanan, begitu seterusnya.\n";
99         cout << "Cara Kerja:\n";
100        cout << "- Syarat mutlak: Data WAJIB diurutkan
(sorting) terlebih dahulu.\n";
101        cout << "- Algoritma akan membelah rentang
pencarian menjadi dua bagian,\n";
102        cout << "  mengecek nilai tengah, lalu membuang
separuh data yang tidak relevan.\n";
103        cout << "Kelebihan:\n";
104        cout << "  * Performa super kilat dan efisien
(kompleksitas  $O(\log n)$ ).\n";
105        cout << "  * Mencari dari 1 juta data maksimal
hanya butuh 20 kali tebakan.\n";
106        cout << "Kekurangan:\n";

```

```

107     cout << " * Tidak bisa langsung dipakai pada
data yang masih acak.\n\n";
108
109     cout << "* KESIMPULAN PENGGUNAAN:\n";
110     cout << "Gunakan Sequential Search jika jumlah
data sedikit atau sering berubah-ubah.\n";
111     cout << "Gunakan Binary Search jika data
berukuran masif dan sudah terstruktur rapi.\n";
112     cout <<
"=====
=====\n";
113 }
114
115 int main() {
116     int opt, target;
117     do {
118         cout << "Pilih menu" << endl;
119         cout << "1. Sequential Searching" << endl;
120         cout << "2. Binary Searching" << endl;
121         cout << "3. Jelaskan Perbedaan Sequential
Searching dan Binary Searching!" << endl;
122         cout << "4. Exit" << endl;
123         cout << "Pilih: ";
124         cin >> opt;
125
126         switch (opt) {
127             case 1: {
128                 vector<int> nums(100);
129                 mt19937_64 rng(random_device{}());
130                 uniform_int_distribution<int>
dist(1, 50);
131
132                 for (auto &val: nums) {
133                     val = dist(rng);
134                 }
135
136                 cout << "100 data acak telah
dibuat:\n";
137
138                 cout << "Data: ";
139                 for (int val : nums) {
140                     cout << val << " ";
141                 }
142                 cout << "\n";

```

```

143         cout << "Masukkan angka yang ingin
dicari: ";
144         cin >> target;
145
146         sequentialSearch(nums, target);
147         break;
148     }
149
150     case 2: {
151         int size;
152         cout << "Masukkan ukuran vector: ";
153         cin >> size;
154
155         vector<int> nums(size);
156         mt19937_64 rng(random_device{}());
157         uniform_int_distribution<int>
dist(1, 100);
158
159         for (auto &val: nums) {
160             val = dist(rng);
161         }
162
163         cout << size << " data acak telah
dibuat:\n";
164         cout << "Data: ";
165         for (int val : nums) {
166             cout << val << " ";
167         }
168         cout << "\n";
169
170         cout << "Masukkan angka yang ingin
dicari: ";
171         cin >> target;
172
173         binarySearch(nums, target);
174         break;
175     }
176
177     case 3:
178         explain();
179         break;
180
181     case 4:
182         cout << "\nTERIMA KASIH\n";

```

```

183         cout << "Program dibuat oleh Aufa
Thariq Akram (2510817310014)" << endl;
184         break;
185
186         default:
187             cout << "Opsi tidak terdefinisi,
mohon masukkan ulang opsi" << endl;
188             break;
189     }
190
191     if (opt != 4) {
192         cout << "\nTekan sembarang tombol untuk
melanjutkan...";
193         getch();
194         clearScreen();
195     }
196
197     } while (opt != 4);
198
199     return 0;
200 }

```

B. Output Program

```

Pilih menu
1. Sequential Searching
2. Binary Searching
3. Jelaskan Perbedaan Sequential Searching dan Binary Searching!
4. Exit
Pilih:

```

Gambar 1 Tampilan Awal Program

```

Pilih menu
1. Sequential Searching
2. Binary Searching
3. Jelaskan Perbedaan Sequential Searching dan Binary Searching!
4. Exit
Pilih: 1
100 data acak telah dibuat:
Data: 34 26 44 8 41 35 45 19 3 23 23 43 22 40 48 22 25 9 30 45 29 32 37 20 44 18 32 9 32 38 41 42 49 24 32 5 45 44 5 22 29 24 19 5 38 41 6 25 25 22 33 19 13 39 44 40 39 15 15 21 30 44 9 45
11 12 9 22 12 17 47 35 8 8 40 42 49 1 33 7 7 30 40 1 6 6 47 19 9 20 50 40 6 11 14 11 30 41 1 27
Masukkan angka yang ingin dicari: 7
Data ditemukan sebanyak 2 kali.
Kemunculan terakhir pada indeks ke-88.
Tekan sembarang tombol untuk melanjutkan...Pilih menu

```

Gambar 2 Sequential Searching dengan Input 7

```

Tekan sembarang tombol untuk melanjutkan...Pilih menu
1. Sequential Searching
2. Binary Searching
3. Jelaskan Perbedaan Sequential Searching dan Binary Searching!
4. Exit
Pilih: 2
Masukkan ukuran vector: 19
19 data acak telah dibuat:
Data: 65 1 25 76 83 86 69 48 30 70 33 93 3 39 71 24 69 65 20
Masukkan angka yang ingin dicari: 71
Data setelah diurutkan:
Data: 1 3 20 24 25 30 33 39 48 65 65 69 69 70 71 76 83 86 93
Angka ditemukan!

```

Gambar 3 Binary Searching Dengan Ukuran Vektor 19 dan Mencari Angka 71

```

Pilih: 3

=====
                KONSEP DASAR SEQUENTIAL & BINARY SEARCHING
=====

[A] SEQUENTIAL SEARCH (Pencarian Linear)
Analogi: Seperti mencari sebuah buku spesifik di dalam tumpukan
        kardus yang berantakan dengan mengeceknya satu per satu.
Cara Kerja:
- Algoritma menyalisir data dari elemen paling awal (indeks 0)
  sampai elemen paling akhir secara berurutan.
Kelebihan:
* Sangat fleksibel! Data boleh dalam kondisi acak atau tidak terurut.
Kekurangan:
* Sangat lambat untuk data berskala besar (kompleksitas  $O(n)$ ).
  Bayangkan mengecek 1 juta data satu per satu dari awal.

[B] BINARY SEARCH (Pencarian Bagi Dua)
Analogi: Seperti mencari nama orang di dalam buku telepon.
        Kita langsung buka bagian tengah, lalu menebak apakah
        namanya ada di paruh kiri atau kanan, begitu seterusnya.
Cara Kerja:
- Syarat mutlak: Data WAJIB diurutkan (sorting) terlebih dahulu.
- Algoritma akan membelah rentang pencarian menjadi dua bagian,
  mengecek nilai tengah, lalu membuang separuh data yang tidak relevan.
Kelebihan:
* Performa super kilat dan efisien (kompleksitas  $O(\log n)$ ).
* Mencari dari 1 juta data maksimal hanya butuh 20 kali tebakan.
Kekurangan:
* Tidak bisa langsung dipakai pada data yang masih acak.

* KESIMPULAN PENGGUNAAN:
Gunakan Sequential Search jika jumlah data sedikit atau sering berubah-ubah.
Gunakan Binary Search jika data berukuran masif dan sudah terstruktur rapi.
=====

```

Gambar 4 Penjelasan Perbedaan Sequential Searching dan Binary Searching

```

Pilih: 4

TERIMA KASIH
Program dibuat oleh Aufa Thariq Akram (2510817310014)
MacBook-Air-AUFA:Downloads aufathariqakram$ █

```

Gambar 5 Exit

C. Pembahasan

baris 1 - 8, syntax `#include` , `#include` , `#include` , `#include` , `#include <termios.h>`, dan `#include <unistd.h>` berfungsi untuk mengintegrasikan kumpulan pustaka atau cetak biru bawaan dari bahasa C++ dan antarmuka sistem operasi berbasis UNIX ke dalam program yang sedang dibangun. pustaka `iostream` secara fundamental mengaktifkan jalur komunikasi aliran data antara memori program dengan perangkat keras layar dan keyboard untuk urusan input-output standar. pustaka `random` menyediakan mesin penghasil angka acak tingkat lanjut yang jauh lebih canggih dan merata distribusinya dibandingkan fungsi `rand()` klasik. pustaka `vector` bertugas untuk memanggil struktur data array dinamis yang kapasitas memorinya bisa membesar atau mengecil secara otomatis saat program berjalan, yang mana ini sangat krusial untuk menampung ratusan data acak nanti. pustaka `algorithm` dipanggil secara spesifik agar program dapat meminjam fungsi `sort()` yang sudah sangat dioptimasi kinerjanya oleh para pengembang C++ untuk mengurutkan data. sementara itu, `termios.h` dan `unistd.h` merupakan pustaka antarmuka sistem operasi tingkat rendah yang menjembatani program dengan driver terminal di macOS agar bisa memanipulasi bagaimana ketikan keyboard diproses secara langsung. adapun syntax `using namespace std;` di baris bawahnya adalah sebuah instruksi direktif kepada kompiler agar program secara otomatis mencari semua fungsi standar di dalam ruang nama `std`, sehingga penulisan kode selanjutnya tidak perlu lagi dikotori dengan awalan `std::` yang berulang-ulang dan memakan waktu.

baris 10 - 21, syntax `int getch()` berfungsi untuk mendefinisikan sebuah fungsi penangkap karakter kustom tingkat rendah yang diciptakan sebagai solusi kompatibilitas karena pustaka `conio.h` milik Windows tidak diakui oleh kernel macOS. fungsi ini beroperasi dengan teknik manipulasi atribut terminal menggunakan struktur data `termios`, di mana ia pertama-tama akan menyalin konfigurasi terminal asli ke dalam variabel `oldt` dan `newt` untuk dicadangkan. setelah itu, ia akan memodifikasi variabel `newt` dengan cara mematikan secara paksa bendera `ICANON` yang mengubah pembacaan dari mode per-baris menjadi mode per-karakter, serta mematikan bendera `ECHO` agar karakter yang ditekan oleh pengguna tidak tercetak atau menempel di layar

terminal. pengaturan baru ini kemudian disuntikkan secara instan ke dalam sistem melalui perintah `tcsetattr` dengan argumen `TCSANOW`, yang memungkinkan fungsi `getchar()` setelahnya dapat langsung bereaksi seketika saat satu tombol keyboard ditekan tanpa harus menunggu tombol enter. sesaat setelah karakter rahasia itu berhasil ditangkap, program akan bergegas melakukan pemulihan sistem dengan mengembalikan konfigurasi terminal ke wujud aslinya menggunakan variabel `oldt` yang sudah disimpan tadi agar terminal tidak mengalami kerusakan atau malfungsi setelah program selesai dijalankan.

baris 23 - 40, syntax `void sequentialSearch(vector &nums, int target)` berfungsi untuk mengkapsulasi blok logika dari algoritma pencarian berurutan yang paling dasar di dalam ilmu komputer. fungsi ini sengaja dirancang untuk menerima wadah data `vector` menggunakan simbol ampersand yang menandakan mode operan `pass-by-reference`, sebuah teknik manajemen memori yang membuat program tidak perlu menduplikasi seluruh isi array yang berjumlah ratusan tersebut ke lokasi memori baru, melainkan cukup membaca alamat aslinya saja sehingga proses komputasi menjadi sangat hemat RAM. mekanisme utamanya digerakkan oleh struktur perulangan `for` yang bertugas menjadi mesin inspektur untuk menelusuri memori, memindai dan membandingkan elemen data satu demi satu secara buta mulai dari titik indeks paling nol hingga menyentuh batas akhir ukuran `vector`. di dalam putaran tersebut, setiap kali mesin mendeteksi adanya kesamaan nilai antara isi elemen dengan angka `target`, ia akan segera memicu penambahan nilai akumulasi pada variabel `counter` sekaligus mencatat rekam jejak posisi indeks terbaru pada variabel `lastIndex`, yang mana ini membuktikan bahwa algoritma linear ini harus terus bekerja secara kejam menelusuri seluruh data hingga habis meskipun `target` sudah ditemukan di awal pencarian, sebelum akhirnya melaporkan rincian data kemunculan tersebut ke layar.

baris 42 - 70, syntax `void binarySearch(vector &nums, int target)` berfungsi untuk menyelenggarakan eksekusi algoritma pencarian bagi dua yang terkenal dengan efisiensi tingkat tingginya. karena algoritma ini memiliki syarat mutlak tidak bisa bekerja pada data yang berantakan, maka langkah pertama yang dilakukannya adalah memanggil kekuatan perintah `sort()` bawaan dari pustaka `algorithm` untuk merapikan

urutan angka di dalam vector dari nilai terkecil hingga terbesar di dalam ruang memori. setelah pondasi data dipastikan rapi dan dicetak ke layar agar pengguna bisa melihat perubahannya, fungsi ini akan mendeklarasikan dua buah variabel penunjuk koordinat yaitu kiri yang menunjuk ke indeks awal dan kanan yang menunjuk ke indeks ujung akhir. inti dari kecepatannya terletak pada struktur perulangan while di mana program akan terus-menerus mencari titik tengah matematis dari rentang koordinat tersebut dan membandingkannya dengan nilai target; jika angka di titik tengah ternyata lebih besar dari target, maka seluruh sisi kanan memori akan langsung dieliminasi secara permanen dari area pencarian, begitu pula sebaliknya. proses pemotongan ruang pencarian menjadi setengah ini akan terus berulang secara eksponensial dalam sekejap mata hingga variabel boolean ketemu berubah statusnya menjadi benar karena target berhasil dijepit, atau hingga titik koordinat kiri dan kanan saling bersilangan yang menandakan bahwa angka target memang tidak pernah eksis di dalam himpunan data tersebut.

baris 72 - 74, syntax void clearScreen() berfungsi untuk menciptakan sebuah modul fungsional yang sangat esensial dalam aspek pengalaman antarmuka pengguna agar terminal tidak dipenuhi oleh lautan teks hasil proses sebelumnya yang menumpuk tak beraturan. fungsi tunggal ini bertindak sebagai sebuah pembungkus sederhana yang memberikan instruksi langsung kepada jeroan sistem operasi inang melalui pemanggilan perintah system("clear"). ketika baris ini dieksekusi, program C++ akan sementara waktu menyerahkan kendali kepada shell UNIX yang mendasari macOS untuk mengirimkan sinyal khusus ke perangkat lunak emulator terminal, memerintahkannya untuk menyapu bersih seluruh baris karakter yang terlihat di layar dan mengembalikan kursor kerdip ke titik koordinat kiri atas, yang mana pendekatan ini secara fungsional merupakan substitusi langsung dari perintah system("cls") yang hanya dikenali di dalam lingkungan sistem operasi Windows.

baris 77 - 105, syntax void explain() berfungsi sebagai modul penyimpanan data string berkapasitas besar yang memuat keseluruhan teks literasi dan penjelasan teoritis mengenai perbedaan paradigma antara kedua algoritma pencarian tersebut. secara teknis, ruang lingkup fungsi ini sama sekali tidak memuat operasi komputasi,

kalkulasi matematis, atau manipulasi memori apa pun, melainkan murni hanya diisi oleh serangkaian panggilan fungsi keluaran `cout` yang telah diformat secara presisi sedemikian rupa dengan menyisipkan berbagai karakter penanda baris baru atau `newline`. modul ini bertindak seperti sebuah buku digital statis yang ketika dipanggil oleh sistem akan langsung menembakkan aliran rentetan karakter ke dalam buffer output standar secara berurutan, menyusun paragraf-paragraf panjang di layar monitor yang menjabarkan analogi pencarian, membedah karakteristik kompleksitas waktu eksekusi dari algoritma linear dan logaritmik, merinci apa saja kelebihan dan batasan teknis dari masing-masing metode, serta memberikan rangkuman pengambilan keputusan rasional bagi pengguna mengenai kapan mereka harus mengimplementasikan masing-masing algoritma tersebut dalam skenario dunia nyata.

baris 107 - 200, `int main()` berfungsi sebagai titik awal absolut dari seluruh siklus kehidupan program atau gerbang utama di mana sistem operasi pertama kali menyerahkan kontrol eksekusi prosesor kepada kode C++ ini. arsitektur di dalamnya dibangun di atas fondasi struktur perulangan `do-while` yang tidak akan pernah berhenti merender tampilan menu interaktif ke layar dan menahan alur program menggunakan `cin` untuk menunggu pengguna mengetikkan instruksi angka. saat opsi dimasukkan, blok percabangan `switch-case` tingkat lanjut akan mengambil alih komando untuk menavigasi rute eksekusi; di mana untuk opsi satu dan dua, program akan membangkitkan mesin generator angka acak tingkat militer bernama `mt19937_64` yang digerakkan oleh bibit entropi murni dari perangkat keras melalui `random_device`, lalu menyaring angka-angka liar tersebut menggunakan objek `uniform_int_distribution` agar rentang angkanya merata dan tidak bias, sebelum akhirnya menuangkan angka-angka acak tersebut untuk mengisi setiap rongga sel memori di dalam struktur `vector` menggunakan mekanisme perulangan rentang berbasis `auto`. setelah data siap saji, barulah ia mendelegasikan beban kerja ke fungsi pencarian yang sesuai dengan pilihan pengguna, lalu akhirnya menampilkan fitur penguncian layar sementara menggunakan fungsi `getch()` yang dibuat sebelumnya agar pengguna bisa membaca hasil dengan tenang sebelum layar dibersihkan ulang, hingga akhirnya pengguna benar-benar

menekan opsi angka empat untuk membunuh proses perulangan dan mengakhiri jalannya program secara terhormat.

LINK GITHUB

<https://github.com/IntraGH/Algoritma-dan-Struktur-Data/tree/main/Modul%206>